

M1 informatique, M2 SAAD
TP. OLAP
Unité YSMIN1A1 : Bases de données avancées

L'objectif de ce TP est d'écrire le schéma d'un cube *OLAP* sur le *datawarehouse* construit dans le TP précédent, ainsi que des requêtes multidimensionnelles permettant de créer des vues sur ce cube.

Si besoin, on pourra utiliser un *datawarehouse* basique, fourni sous la forme d'une base de données SQLite (`basic_datawarehouse.db`; utiliser SQLite pour en voir le schéma). Ce *datawarehouse* est déjà construit et alimenté. Il définit la structure et les données nécessaires à l'analyse, mais ne définit aucun des éléments qui permettrait la traçabilité des données ou la prise en compte de l'évolution des dimensions. Il est ainsi plus intéressant de travailler avec le *datawarehouse* construit dans le TP précédent, s'il est prêt.

Note Le fichier `basic_datawarehouse.db` est fourni dans l'archive `tp_olap.zip` (voir la partie 1).

1 Prise en main de Pentaho Mondrian

Le schéma et les requêtes seront testés en utilisant Pentaho Mondrian, en ligne de commande.

Manipulation 1. *Lancer une première exécution de Pentaho Mondrian :*

- désarchiver le fichier `tp_olap.zip` fourni;
- se rendre dans le répertoire `tp_olap`;
- exécuter (sur une seule ligne)

```
sh run.sh -p examples/example.properties -f examples/example_query_1.mdx  
| java -jar lib/mondrian_view.jar
```

Cette dernière commande lance une requête écrite dans le fichier `examples/example_query_1.mdx`, sur le *datawarehouse* stocké dans la base de données renseignée dans `examples/example.properties`, et selon le cube dont le schéma est renseigné dans ce même fichier. En l'occurrence, l'exemple donné utilise le *datawarehouse* basique évoqué plus haut (`datawarehouse/basic_datawarehouse.db`) avec le cube défini dans `examples/example_schema.xml`.

De façon générale, cette commande affiche le résultat de la requête de façon graphique; si on enlève l'appel à `java`, on obtient le résultat brut (en mode texte) du `CmdRunner` de Mondrian.

2 Spécification du cube et requêtes MDX

Dans le reste de ce TP, il s'agit de compléter le schéma donné en exemple, et d'écrire un ensemble de requêtes MDX. De façon générale, la spécification du cube peut être effectuée indépendamment des requêtes, en envisageant les cas d'utilisation possibles. Toutefois, dans le cadre de ce TP, on pourra enrichir la spécification au fil de l'écriture des requêtes.

Les exemples de schéma et de requêtes fournis dans les fichiers sont destinés à servir de modèle. Quelques précisions sur la spécification des schémas et sur le langage MDX sont également données plus loin (en partie 3). Pour la suite, afin de conserver les exemples en l'état, créer une copie du répertoire `examples` et de ses fichiers (en nommant la copie, par exemple, `tp`, en renommant ses fichiers en `schema.xml`, etc.), et en adaptant le fichier `.properties`. Travailler ensuite exclusivement sur ces nouveaux fichiers.

Exercice 2. Écrire une requête MDX permettant de visualiser le chiffre d'affaires total (somme des prix des ventes), sur l'ensemble des ventes.

Exercice 3. Écrire une requête permettant de visualiser le chiffre d'affaires total par mois, les mois étant affichés en toutes lettres. Qu'observe-t-on en exécutant la requête ?

Exercice 4. Écrire une requête permettant de visualiser le chiffre d'affaires total par mois, les mois étant affichés en toutes lettres, pour les prestations réalisées en Normandie et pour celles réalisées en Bretagne.

Exercice 5. Écrire une requête permettant de visualiser le chiffre d'affaires total par mois, les mois étant affichés en toutes lettres et les ventes étant cumulées seulement sur celles ayant été réalisées en Normandie et en Bretagne.

Exercice 6. Écrire une requête permettant de visualiser le chiffre d'affaires et le nombre de ventes par heure de la journée (cumulés sur toute la période). Par exemple, on voudrait pouvoir visualiser le nombre total de ventes réalisés à 4 h du matin, sur tout l'historique. Qu'observe-t-on en exécutant la requête ?

Indication : Il faudra définir des hiérarchies dédiées.

Exercice 7. Écrire de même une requête pour les jours du mois. Par exemple, on voudrait pouvoir visualiser le nombre total de ventes réalisés un 15 du mois, sur tout l'historique. Qu'observe-t-on en exécutant la requête ?

Exercice 8. Écrire une requête permettant de visualiser les nombres de ventes, en fonction de la région du client et de celle où la prestation a été réalisée. Qu'observe-t-on en exécutant la requête ?

Exercice 9. Écrire une requête permettant de visualiser le nombre et le prix moyen des ventes par département de réalisation de la prestation.

Exercice 10. Écrire une requête permettant de visualiser le nombre et le prix moyen des prestations réalisées en Normandie, et le nombre et le prix moyen de celles réalisées en Bretagne, pour chaque trimestre de 2020. Qu'observe-t-on en exécutant la requête ?

Exercice 11. Écrire une requête permettant de visualiser les durées moyennes d'intervention, en fonction de la catégorie de prestation, de la région où la prestation a été réalisée, et du trimestre. Qu'observe-t-on en exécutant la requête ?

3 Quelques précisions sur les schémas et sur MDX

Concernant la définition du schéma :

- la balise **Table** est utilisée pour spécifier la table des faits ;
- la balise **Dimension** permet de spécifier une dimension, l'attribut **foreignKey** indiquant quel attribut de la table des faits sert de clef (étrangère) vers cette dimension ;
- une dimension peut contenir une ou plusieurs hiérarchies, qui peuvent être nommées pour lever les ambiguïtés (attribut **name** de la balise **Dimension**) ;
- au sein d'une hiérarchie, on spécifie tout d'abord la table dans laquelle est stockée la dimension¹ ;
- toujours au sein d'une hiérarchie, on liste les niveaux, du plus grossier au plus fin ; la spécification **uniqueMembers="true"** indique que chaque membre du niveau n'apparaît que dans un membre du niveau supérieur ; par exemple, dans la hiérarchie naturelle, les villes sont *unique members*, car une ville appartient à un unique département ; en revanche, les mois ne le sont *pas*, car même si le mois de janvier semble apparaître toujours dans le même trimestre (trimestre de numéro 1), il apparaît en réalité dans plusieurs trimestres différents (le trimestre 1 de 2020, le trimestre 1 de 2021, etc.) ;
- pour une hiérarchie, on positionne l'attribut **hasAll** à **true** ou **false** ; **true** signifie que l'on peut agréger sur toutes les valeurs de la dimension correspondante (on n'a en général aucune raison de le positionner à **false**) ;
- on peut spécifier que les membres d'un niveau doivent être triés selon la valeur d'une autre colonne (par exemple les noms de mois en fonction du numéro de mois) en utilisant l'attribut **ordinalColumn** pour le niveau à trier ;
- toute mesure appartient implicitement à la dimension **Measures**.

Concernant le langage MDX :

1. Pour un schéma en flocon, on définit certaines dimensions par des jointures ; il faut alors utiliser la balise **Relation** à la place de **Table** dans la définition de la hiérarchie.

- un ensemble de membres est spécifié entre accolades, les accolades peuvent toutefois être omises pour un ensemble singleton,
- une dimension, une hiérarchie, un niveau, un membre sont spécifiés entre crochets,
- le mot réservé MEMBERS désigne l'ensemble de tous les membres du niveau auquel il est appliqué,
- le mot réservé CHILDREN désigne l'ensemble de tous les enfants (membres contenus, au niveau immédiatement inférieur) du membre auquel il est appliqué,
- de façon générale, les membres des niveaux doivent être spécifiés de manière non ambiguë ; ainsi, on peut écrire `Dimension.Hiérarchie.Niveau.membre`, mais si ce n'est pas ambigu, on peut aussi écrire simplement `membre`, ou encore `Hiérarchie.membre`,
- le *slicer* est spécifié comme un tuple, entre parenthèses,
- COLUMNS et ROWS sont des alias pour les deux premiers axes.